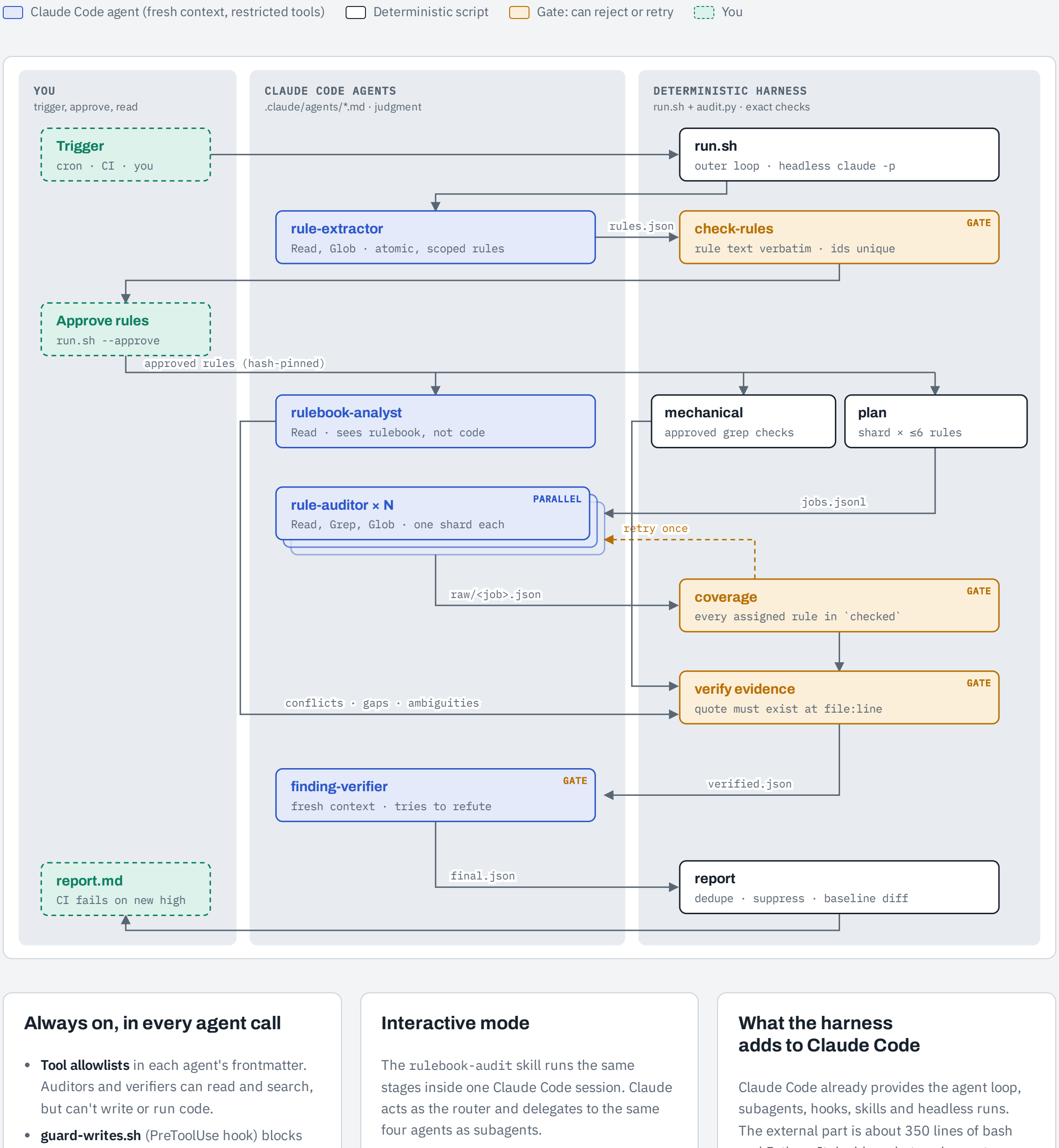
CLAUDE CODE · NO MCP · SKILLS, SUBAGENTS, HOOKS, CLI

Rulebook Audit Loop

A repository is checked against a rulebook for code that breaks rules, and the rulebook itself is checked for conflicts, gaps and ambiguity. Work alternates between agents that judge and scripts that verify. No agent's output is trusted until a script or a second agent has checked it.



Always on, in every agent call

- **Tool allowlists** in each agent's frontmatter. Auditors and verifiers can read and search, but can't write or run code.
- **guard-writes.sh** (PreToolUse hook) blocks any write outside `.audit/`. The audit can't change what it audits.
- **settings.json** denies push, commit, rm, curl, and reads of `.env` and secrets.
- **--json-schema** makes each agent's output parseable. **--max-budget-usd** caps spend per call.

Interactive mode

The `rulebook-audit` skill runs the same stages inside one Claude Code session. Claude acts as the router and delegates to the same four agents as subagents.

- **stop-gate.sh** (Stop hook) refuses to let the session finish while any planned job lacks output or skipped a rule.
- The script gates are the same: `audit.py coverage`, `verify` and `report`.

What the harness adds to Claude Code

Claude Code already provides the agent loop, subagents, hooks, skills and headless runs. The external part is about 350 lines of bash and Python. It decides what each agent sees, checks outputs mechanically, keeps state between runs (baseline, suppressions, fingerprints) and runs on a trigger.

File map

FILE	ROLE	WHAT IT DOES
.claude/		
agents/rule-extractor.md	AGENT	Splits the rulebook into single-obligation rules with verbatim text, file scopes and optional grep checks.
agents/rulebook-analyst.md	AGENT	Finds conflicts, gaps and ambiguities using only the rulebook.
agents/rule-auditor.md	AGENT	Checks one shard against a few rules. Must quote evidence and list every rule it checked.
agents/finding-verifier.md	AGENT	Re-reads code and rules and tries to refute each finding. Confidence below 0.4 drops it.
skills/rulebook-audit/SKILL.md	SKILL	Entry point: batch via run.sh, or run the stages interactively.
hooks/guard-writes.sh	HOOK	PreToolUse: blocks writes outside `.audit/`.
hooks/stop-gate.sh	HOOK	Stop: blocks finishing while coverage is incomplete.
settings.json	CONFIG	Permissions and hook wiring.
scripts/audit/ · schemas/		
run.sh	SCRIPT	Outer loop. Each stage is one headless `claude -p` call that loads the agent file, with parallel fan-out and retry.
audit.py	GATES	`check-rules`, `plan`, `coverage`, `verify`, `report`. No model, no dependencies.
findings.schema.json · rules.schema.json	SCHEMA	Enforced on agent output through `--json-schema`.